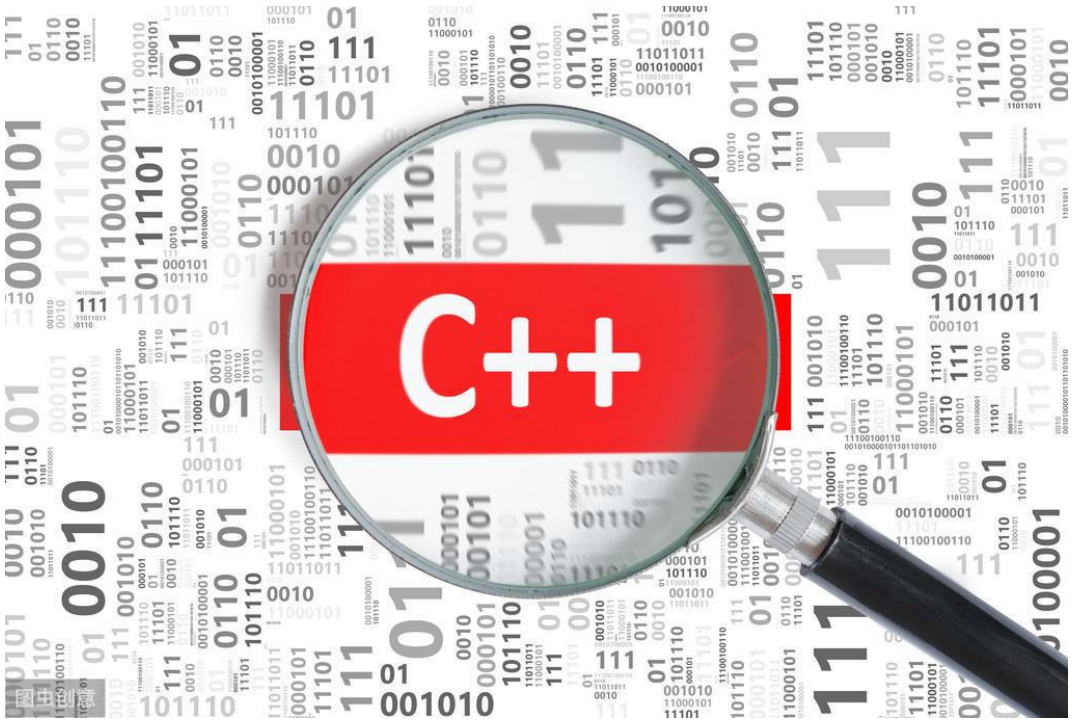

Haskell
JavaScript
Perl
C++
Ruby
Java
Objective-C
Pascal
Assembly
Scheme
MATLAB
Forth
PL/SQL
Lisp
Visual Basic
Mathematica
ColdFusion
FoxPro
Swift

C++语言程序设计


perl
html
C++
sql
php
java
xml
python
delphi
ruby


C++

面向对象程序设计

运算符重载



什么是运算符

西南交通大学 Southwest Jiaotong University

C++已有的定义实现了对数据进行运算的操作符：

① 算术运算符：+、-、*、/、%、++、--

② 位操作运算符：&(与)、|(或)、~(非)、^(异或)、<< (左移)、>>(右移)

③ 逻辑运算符：!、&&、||

④ 比较运算符：<、>、<=、>=、==、!=

⑤ 赋值运算符：=、+=、-=、*=、/=、%=、&=、|=、^=、<<=、>>=

⑥ 其他运算符：

[]、()、->、,(逗号运算符)、new、delete、new[]、delete[]、->*

C++实现了基本的数据类型的运算：整型，实型，字符型，逻辑型……

自定义数据类型与运算符重载

设$z_1=a+bi$，$z_2=c+di$是任意两个复数，

则它们的和是 (a+bi)+(c+di)=(a+c)+(b+d)i

```
class Complex
{
public:
    Complex(double r=0.0,double i=0.0)
    {real=r; imag=i;}
    void display();
private:
    double real;
    double imag;
};
```

自定义数据类型与运算符重载

实例化复数类的对象：

```
Complex a(10,20),b(5,7),c;
c=a+b;
```

如果要将a和b相加，表达式a+b在编译时会出错，因为编译器不知道如何处理这个加法。这时候就需要自己编写程序来说明“+”在作用于complex类对象时，该实现什么样的功能，这就是**运算符重载**。



运算符重载函数定义

西南交通大学 Southwest Jiaotong University

- 重载为非成员函数

关键字

形参表中给出重载运算符所需要的参数和类型

语法形式：

```
函数类型 operator 运算符(形参表)
{
    函数体
}
```

指定重载运算符的名称

如：+ - * /

运算符重载函数定义


```
class Complex{
public:
Complex(double r = 0.0,
double i= 0.0){
     real = r;
     imag = i;
     }
double real;//实部
double imag;//虚部
};
```

相当于调用：c=operator+(a,b)

```
int main()
{
     Complex a(1,2), b(2,3),c;
     c = a + b;
}
```

运算符重载为普通函数

```
Complex operator+ (const Complex & a, const  Complex & b)
{
     return Complex( a.real+b.real, a.imag+b.imag);
}                    // “类名(参数表)”就代表一个对象
```

运算符重载函数定义

```
class Complex
 {
public:
    Complex( double r= 0.0, double m = 0.0 ):  real(r),
    imag(m){}// 构造函数
    Complex operator+ ( const Complex & ); //加 法
    Complex operator- ( const Complex & ); //减 法
private:
    double real;//实 部
    double imag;//虚 部
};
```

运算符重载为成员函数

☞ 重载为成员函数时，参数个数为运算符目数(操作数个数)减一



★运算符重载函数定义


```
// 重载加号运算符
Complex Complex::operator+(const Complex & b)
{
    return Complex( real + b.real, imag + b.imag );
}
// 重载减号运算符
Complex Complex::operator- (const Complex &b)
{
    return Complex( this->real - b.real, this->imag - b.imag );
}
```

这里的real,imag是当前this指针所指对象的real，imag。

也可写成：

- this->real
- this->imag

所以：(*this)就是重载为普通函数时的参数对象a。

```
int main()
{
    Complex x, y(4.3, 8.2), z(3.3, 1.1);
    x=y+z;
    x=y-z;
}
```

```
x=y.operator+(z);
x=y.operator-(z);
```

★运算符重载函数定义

练习： 编写B类的构造函数、拷贝构造函数、重载运算符+和getz

```
class A
{
   int x, y;
public:
   A(int x1 = 0, int y1 = 0) :x(x1), y(y1) {}
   A(const A& a)
   { x = a.x; y = a.y;}
   A operator+(A& a)
   { return A(x + a.x, y + a.y);}
   int get_x()
   { return x;}
   int get_y()
   { return y;}
};
```

```
class B :public A
{
      int z;
public:
      B(int x1 = 0, int y1 = 0, int z1 = 0);//B类构造函数
      B(B& b);  //拷贝构造函数
      B operator +(B& b);  //运算符重载
      int getz();
};
```



自加/自减运算符的重载

西南交通大学 Southwest Jiaotong University

自加++/自减-- 运算符有 前置/后置 之分

(1) 前置运算符作为一元运算符重载

- 重载为类的成员函数:

  T & operator++();

  T & operator--();

- 重载为全局函数:

  T & operator++(T &);

  T & operator--(T &);

T obj;

++obj;

obj.operator++(); operator++(obj);

都调用重载运算符函数

问题(1) 前置++或--的返回类型是什么?

自加/自减运算符的重载

(2) 后置运算符作为二元运算符重载：多写一个参数，无实际意义

- 重载为类的成员函数：

```
T operator++(int);
T operator--(int);
```

- 重载为全局函数：

```
T operator++(T &,int);
T operator--(T &,int);
```

```
T obj;
obj++;
obj.operator++();
operator++(obj);
```

都调用重载运算符函数

问题(2) 后置++或--的返回类型是什么?



自加/自减运算符的重载

西南交通大学 Southwest Jiaotong University

```
int main()
{
    CDemo d(5);
    //若++重载为成员函数
    cout << (d++) <<",": //等价于d.operator++(0);
    cout << d << ",";
    cout << (++d) << ",";//等价于 d.operator++();
    cout << d << endl;
    //若--重载为全局函数
    cout << (d--) << ",";//等价于 operator--(d,0);
    cout << d << ",";
    cout << (--d) << ",";//等价于 operator--(d);
    cout << d << endl;
    return 0;
}
```

运行结果

```
5,6,7,7
7,6,5,5
```



如何编写 CDemo类?

自加/自减运算符的重载

运算符重载为：**成员函数**

```
class CDemo
{
private :
    int n;
public:
    CDemo(int i=0):n(i) { }
    CDemo & operator++();//(1)
    CDemo operator++(int); //(2)

    operator int ( )
    { return n; }
    //(3) 前置--，全局函数 friend
    friend CDemo & operator--(CDemo &);
    //(4) 后置--，全局函数 friend
    friend CDemo operator--(CDemo &,
    int);
};
```

```
//前置 ++
CDemo & CDemo::operator++()
{
    n++;
    return * this;
}
//后置++
CDemo CDemo::operator++(int k)
{
    //记录修改前的对象
    CDemo tmp(*this);
     n++;
    //返回修改前的对象
    return tmp;
}
```

(1)(2)关注返回值类型，具体函数代码写法

(3)(4)关注函数代码写法，如何实现?

★运算符重载为友元函数

西南交通大学
Southwest Jiaotong University

运算符也可以重载为类的友元函数，这样，它就可以自由访问该类的任何数据成员。

这时，运算所需要的**操作数都需要通过函数的形参表来传递**。

语法形式：

```
friend 函数类型 operator 运算符(形参表)
{
    函数体
}
```

关键字

指定重载运算符的名称
如：+ - * /

形参从左到右的顺序**就是**
运算符操作数的顺序

★运算符重载为友元函数

运算符重载为：普通函数(采用友元函数访问私有成员变量)

```
CDemo & operator--(CDemo & d)
{
    d.n--;
    return d;
}
CDemo operator--(CDemo & d, int)
{
    CDemo tmp(d);
    d.n--;
    return tmp;
}
```



★运算符重载为友元函数

需要将运算符重载为友元函数的情况

- 成员函数不能满足使用要求
- 需要访问类的私有成员

```
Complex c ;
c = c + 5;//有定义，相当于
//c = c.operator +(5);
```

但是

```
c = 5 + c;//编译出错
```

为了使得上述表达式能成立，需要将+重载为普通函数。

若有定义，则c = 5 + c;

相当于c =operator +(5,c);

```
class Complex
{
    double real, imag;
public:
    Complex(double r, double i):real(r), imag(i){ }
    Complex operator+(double r);
};
Complex Complex::operator+(double r) //能解释c+5
{
    return Complex(real + r, imag);
}
Complex operator+ (double r, const Complex & c)
 {//能解释      5+c
    return Complex( c.real + r, c.imag);
 }
```

还有什么没有考虑到的问题?



★运算符重载为友元函数

因为普通函数不能访问类的私有成员，所以需将运算符+重载为友元函数。

```
class Complex
{   double real, imag;
public:
    Complex( double r, double i):real(r),imag(i){ };
    Complex operator+( const Complex & b );//能解释c=a+b
    Complex operator+( double r );//能解释c+5
    friend Complex operator + (double r, const Complex & c);//能解释5+c
};
```

```
Complex a(1,2),b(2.3),c;
c=a+b;//c=operator+(a,b)
```

```
CDemo d;
d--;//operator--(d,0);
```

- 友元函数中，运算符**左操作数**与函数第一个参数对应，**右操作数**和第二个参数对应。
- 当重载友元函数时，将没有隐含的参数this指针
  - ✓ 对双目运算符，友元函数有两个参数
  - ✓ 对单目运算符，友员函数有一个参数(后置运算符除外)。

★运算符重载的注意事项

西南交通大学
Southwest Jiaotong University

(1)C++不允许定义新的运算符，重载后运算符的含义应符合日常习惯。

```
complex_a + complex_b
word_a > word_b
date_b = date_a + n
```

(2)重载运算符()，[]，->或者赋值运算符=时，重载函数必须声明为类的成员函数。

(3)重载之后的运算符不能改变运算符的优先级和结合性，也不能改变运算符操作数的个数及语法结构。

(4)重载运算符的函数性质符合函数重载，但是不能有默认的参数。

(5)用户自定义类的运算符一般都必须重载后方可使用，但有个例外，运算符=不必重载。



★运算符重载的注意事项

(6) C++中只有5种运算符不能重载(. .* :: sizeof ?:)

① .类属关系运算符号(以保证C++中访问成员功能的含义不被改变)

② .*成员指针运算符(以保证C++中访问成员功能的含义不被改变)

③ ::作用域分辨符(操作数是类型，而不是普通的表达式，不具备重载特征，如：d1.B0::fun())

④ sizeof运算符(操作数是类型，而不是普通的表达式，不具备重载特征)

⑤ ?:条件运算符，唯一的一个三目运算符

(7) C++中必须重载为成员函数的运算符

赋值=、下标[]、函数调用()和成员访问箭头->运算符必须重载为成员(函数)。

(8) C++中必须重载为友元(普通函数)的运算符

① 输入>>、输出<<运算符

② 第一个操纵算子非类型成员必须重载为友元(函数)。

★特殊运算符重载

西南交通大学 Southwest Jiaotong University

特殊运算符包括：

赋值运算符 、 下标运算符、 强制类型转换、 关系运算符

1. 重载：赋值运算符 =

- 赋值运算符 两边的类型 可以不匹配。如：
  - 把一个int类型变量赋值给一个 Complex对象
  - 把一个char * 类型的字符串赋值给一个字符串对象
- 需要重载赋值运算符=
- 赋值运算符=只能重载为 成员函数



★特殊运算符重载

思考：编写一个长度可变的字符串类String



(1) 包含一个char * 类型的成员变量

(2) 每个对象指向动态分配的存储空间

(3) 该存储空间用于存放'\0'结尾的字符串

```
class String
{
private:
    char * str;
public:
    String () : str(NULL) { } //构造函数，初始化str为NULL
    const char * c_str() { return str; }
    char * operator = (const char * s);
    ~String( );
};
```

★特殊运算符重载

//重载 '=' 使得obj = "Good Luck"能够成立

```
char * String::operator = (const char * s)
{
    if(str) delete [] str;
    if(s)
        {//s不为NULL才执行拷贝
            str = new char[strlen(s)+1];
            strcpy(str, s);
        }
    else
        str = NULL;
    return str;
}
```

```
String::~String( )
{
    if(str)
        delete [] str;
}
int main()
{
    String s;
    s = "Good Luck.";
    cout << s.c_str() << endl;
// String s2 ="Hello!"; //这条语句要是不注释掉就会出错。为什么?
}
```

★赋值=运算符的重载

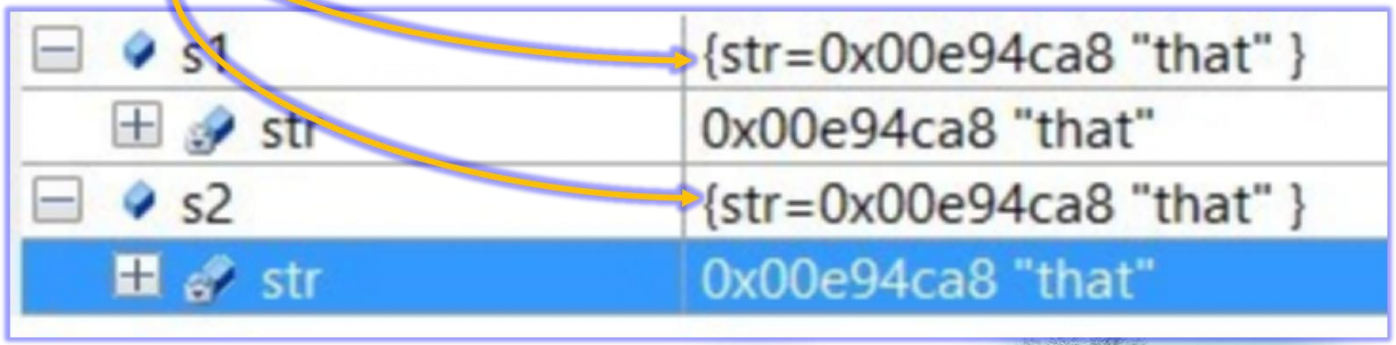
重载赋值运算符的意义—浅拷贝和深拷贝

S1 = S2;

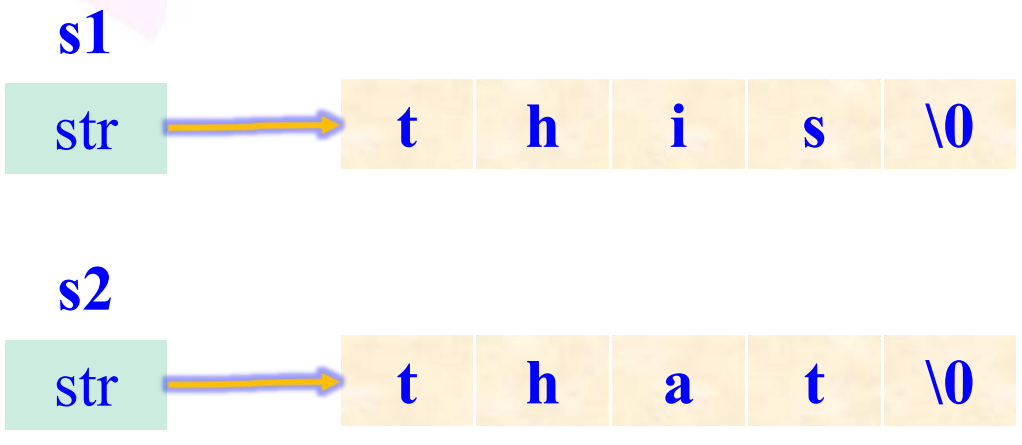
① 浅复制/浅拷贝

- 共享内存地址
- 执行逐个字节的复制工作

```
String s1,s2;
s1 = "this";
s2 = "that";
s1 = s2;
```

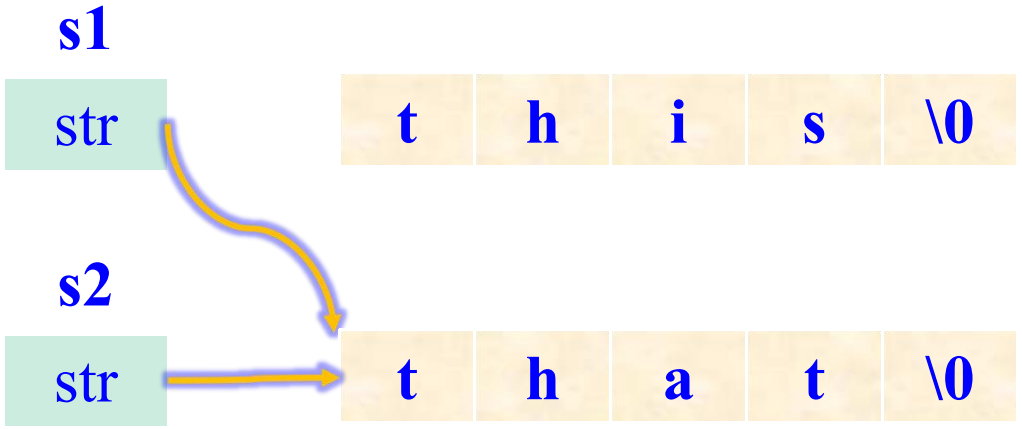


★特殊运算符重载




```
String s1,s2;
s1 = "this";
s2 = "that";
s1 = s2;
```

浅复制/浅拷贝




```
s1 = s2;
```

★赋值=运算符的重载

西南交通大学
Southwest Jiaotong University

② 深复制/深拷贝

将一个对象中指针变量指向的内容，

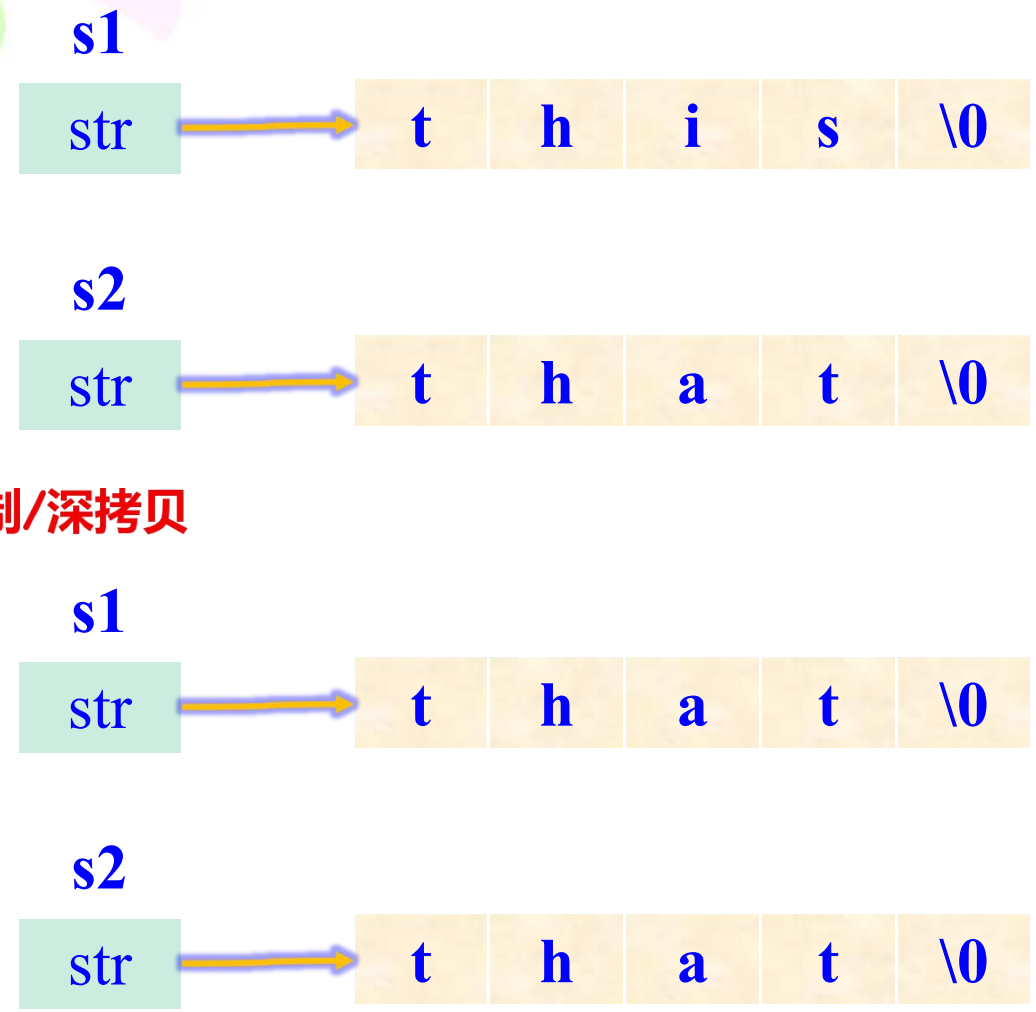
复制到另一个对象中指针成员对象指向的地方

```
String s1,s2;
s1 ="this";
s2 ="that";
s1 =s2;
```

在class String 里添加成员函数:

```
String & operator = (const String & s)
{
    if(str) delete [] str;
    str = new  char[strlen(s.str)+1];
    strcpy(str, s.str);
    return * this;
}
```

★特殊运算符重载




```
String s1,s2;
s1 = "this";
s2 = "that";
s1 = s2;
```

```
s1 = s2;
```

★特殊运算符重载



```
String s1;
s1 ="this";
s1=s1;
```

是否会有问题?

正确写法:

```
String & String::operator = (const String & s)
{
    if(str == s.str) return * this;//处理s1=s1
    if(str) delete [] str;
    if(s.str)
        { //s.str不为NULL才会执行拷贝
            str = new char[strlen(s.str)+1];
            strcpy( str,s.str);
        }
    else
        str = NULL;
     return * this;
}
```

★特殊运算符重载

重载赋值运算符=

对 operator = 返回值类型的讨论

(1) void 好不好?

考虑: a = b = c;

//等价于a.operator=(b.operator=(c));

(2) String 好不好?

(3) 为什么是String &? 因为赋值运算的结果会作为赋值运算的左值!

运算符重载时，好的风格——尽量保留运算符原本的特性

考虑: (a=b)=c; //会修改a的值

等价于: (a.operator=(b)).operator=(c);

★特殊运算符重载

请问上面的String类是否就没有问题了？

为String类编写复制构造函数时会面临和=同样的问题，需用同样的方法处理

```
String::String(String & s)
{
    if(s.str)
    {
        str = new char[strlen(s.str)+1];
        strcpy(str, s.str);
    }
    else
    str = NULL;
}
```

★特殊运算符重载

2. 下标运算符 []重载示例

```
class Demo
{
private:
    int Vector[5];
public:
    Demo() {};
    int &operator[ ](int i)
    {
        return Vector[i];
    }
};
```

```
int  main()
{
    Demo v;
    for(int i=0;i<5;i++)
        v[i]=i+1;
    for(i=0;i<5;i++)
        cout<<v[i]<<" ";
    cout<<endl;
    return 0;
}
```

运行结果

```
1 2 3 4 5
```

★特殊运算符重载

3. 类型强制转换运算符——只能重载为成员函数

```
operator int ( )
{
return n;
}
```

int 作为一个类型强制转换运算符被重载：

```
Demo s;
(int) s ;
```

- 类型强制转换运算符重载时：
  - 不能写返回值类型
  - 实际上其返回值类型就是类型强制转换运算符代表的类型

★特殊运算符重载

4. 关系运算符重载

==, !=, >, <, >=, <=

```
class String
{
private:
        char * str;
public:
        String ():str(NULL) { }
        bool operator==(const String &str1) const;
        bool operator!=(const String &str1) const;
        bool operator>(const String &str1) const;
        bool operator<(const String &str1) const;
        bool operator>=(const String &str1) const;
        bool operator<=(const String &str1) const;
        …
}
```

★特殊运算符重载


```
bool String::operator==(const String &str1) const
    {return !(strcmp(this->str,str1.str));}
bool String::operator!=(const String &str1) const
    {return !(*this==str1);}
bool String::operator>(const String &str1) const
    {if(strcmp(this->str,str1.str)>0)  return true;}
bool String::operator<(const String &str1) const
    {return str1>*this;}
bool String::operator>=(const String &str1) const
    {return !(str1>*this);}
bool String::operator<=(const String &str1) const
    {return !(*this>str1);}
```

★特殊运算符重载


```
int main(){
    String s1,s2;
    s1 = "this";
    s2 ="that";
    cout << s1.c_str() << endl;
    cout << s2.c_str() << endl;
    if(s1==s2)
        cout<<"s1==s2"<< endl;
    else if (s1!=s2)
        cout<<"s1!=s2"<< endl;
    if(s1>=s2)  cout<<"s1>=s2"<< endl;
    else if(s1<s2)  cout<<"s1<s2"<< endl;
    return 0;
}
```

运行结果

```
this
that
s1!=s2
s1<s2
```

★运算符重载归纳

- 主要优点：允许改变使用于系统内部的运算符的操作方式，以适应用户自定义类型的类似运算
- 方法：重载为类的成员函数和重载为类的友元函数
- 区别：
  - 当运算符重载为类的成员函数时，函数的参数个数比原来的操作数个数要少一个(后置运算符除外)。
  - 当重载为类的友元函数时，参数个数与原操作数个数相同。



面向对象程序设计

多态性



★多态性概述

📖多态：相同对象收到不同消息

同样的消息被不同类型的对象接收

产生完全不同的行为

📖消息：指对类的成员函数的调用。

📖不同的行为：指不同的实现，也就是调用了不同的函数。



★多态性概述

1.静态多态

函数多态

普通函数的重载及类的成员函数的重载，以及运算符重载

宏多态

带变量的宏多态可实现一种初级形式的静态多态。

```
#define __ADD(A,B) (A)+(B)
//定义泛化的宏__ADD
```

模板多态

与类模板相关联，在使用时赋予实际的类型将类模板实例化。这样，由类模板实例化的各个类都具有相同的操作，而操作对象的类型则各不相同。

例：模板多态

```
#include<iostream>
#include<algorithm>
#include<iomanip>
#include<string>
using namespace std;
const int N=3;
class CBase{
        public:
                CBase():age(0),name(""){}
                CBase(int x,string s=""):age(x),name(s){}
                void print()    {
                        cout<<age<<","<<name<<endl;
                }
                friend int cmp(const CBase &a,const CBase &b);
        private:
                int age;
                string name;
};
```

例：模板多态(续)

```
int cmp(const CBase &a,const CBase &b)
{    return a.age>b.age;}
int cmp1(int a,int b)
{    return a>b;}
int main() {
        int x[N]={53,44,66};
        sort(x,x+N,cmp1); //对整型数组排序
        for(int i=0;i<N;i++)
                cout<<setw(3)<<x[i];
        cout<<endl<<endl;
        CBase a[N]={CBase(45,"Zhang san"),CBase(34,"Li si"),
                        CBase(58,"Wang wu")       };
        sort(a,a+N,cmp); //对 CBase对象数组排序
        for(int i=0;i<N;i++)
                a[i].print();
        return 0;
}
```

运行结果

```
 66 53 44

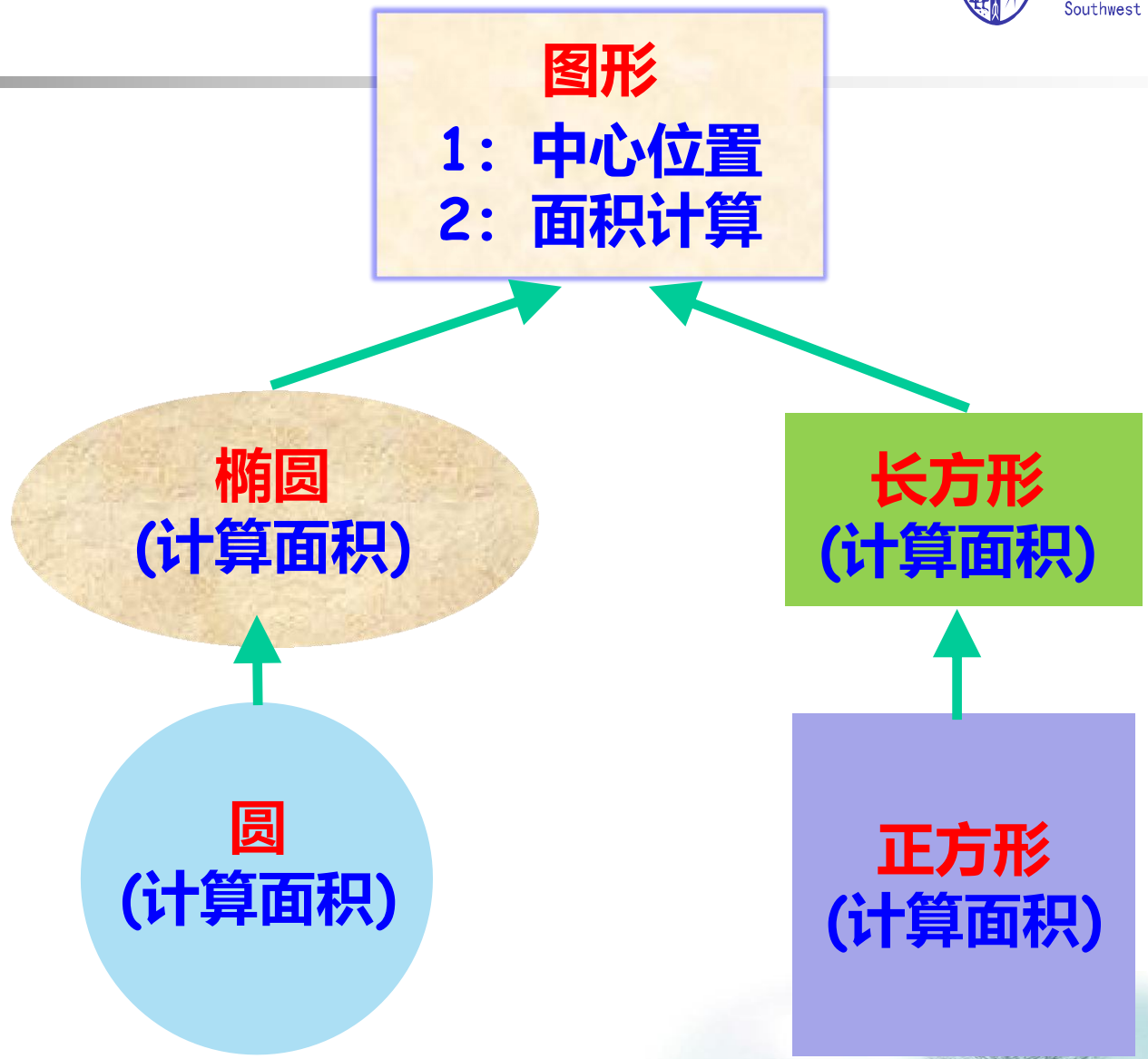
58,Wang wu
45,Zhang san
34,Li si
```

★多态性概述

2.动态多态

- C++的动态多态性是C++实现面向对象技术的基础。

具体的说，通过一个指向基类的指针调用虚成员函数的时候，运行时系统将能够根据指针所指向的实际对象调用恰当的成员函数。



确定图形中心位置，求面积

★多态实现规则

1.赋值兼容规则

指在需要基类对象的任何地方都可以使用公有派生类的对象来替代。在替代之后，派生类对象就可以作为基类的对象使用，但只能使用从基类继承的成员。

```
class A {};
class B : public A{};
int main()
{
    A a;
    B b;
    a=b;
    A &r_a=b;
    A *p_a=&b;
    return 0;
}
```

向上强制转换规则(is a)，不需要进行显示转换：

(1) 派生类的对象可以赋值给基类对象

```
a = b;
```

(2) 派生类对象可以初始化基类引用

```
A & r_a = b;
```

(3) 派生类对象的地址可以赋值给基类指针

```
A* p_a = &b;
```

注意：如果派生方式是 private或protected，则上述三条不可行

★多态实现规则

例4a：通过基类指针只能访问到基类的成员

```
#include <iostream>
using namespace std;
class A
{
public :
        void print( ) {   cout << "A::print"<<endl ; }
};
class B:public A
{
public :
        void print( ) {   cout << "B::print" <<endl; }
};
```

```
int main( )
{
     A a;
     B b;
     A *pA = &b;
      pA->print( );
     return 0;
}
```

运行结果

```
A::print
```

- 根据**赋值兼容规则**，使用基类指针**指向派生类**，就可以**通过这个指针**来使用**派生类的成员函数**，但若此函数是**普通的成员函数**，通过基类指针访问到的只能是**基类的成员**。

3.多态的实现

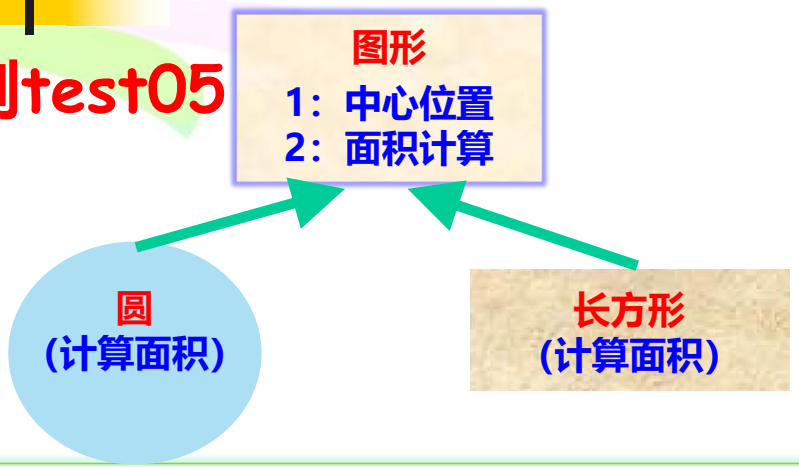
1．静态联编(早期联编，前联编)：联编工作在编译连接阶段完成即为静态联编。

2．动态联编(晚期联编，后联编)：联编工作在程序运行阶段完成即为动态联编。



★多态实现规则

例test05




```
class Center
{
private:
    double x,y;
public:
    Center(){x=0,y=0;}
    ~Center(){cout<<"~中心"<<endl;}
    void SetCenter(double x1,double y1)
    {x=x1;y=y1;}
};
```

```
class CShape
{
private:
    Center c1;
public:
    void area()
    {cout<<"计算面积:"<<endl;}
    ~ CShape()
    {cout<< "~图形"<<endl;}
    void setC(double x1,double y1)
    { c1.SetCenter(x1,y1);}
};
```

★多态实现规则

图形
1：中心位置
2：面积计算

圆
(计算面积)

长方形
(计算面积)

```
class Circle:public CShape
{
private:
    double radius;
public:
    Circle(double r):radius(r){}
    ~Circle (){cout<< "~圆 "<<endl;}
    void area()//圆计算面积
    {
    cout<< 3.14*radius*radius<<endl;
    }
};
```

```
class CRetangle:public CShape
 {
 private:
     double width,height;
 public:
     CRetangle(double w1,double h1)
     { width=w1; height=h1; }
     ~CRetangle ()
     {cout<<"~长方形"<<endl;}
     void area()//计算长方形面积
     {cout<<width*height<<endl;}
};
```

★多态实现规则

```
int main()
{
    CShape *s1=new CShape;//动态创建图形对象
    s1->area();//调用计算图形面积函数
    Circle  *c1=new Circle(2.0);//动态创建图形对象
    s1=c1;//图形指针s1指向圆指针c1指向的圆对象
    s1->area();//调用计算面积函数?
    CRetangle *r1=new CRetangle(2.0,3.0); //动态创建长方形形对象。
    s1=r1;//图形指针s1改为指向长方形指针r1指向的长方形对象
    s1->area();//调用计算面积函数?
    return 0;
}
```



★多态实现规则

运行结果：

西南交通大学 Southwest Jiaotong University

```
int main()
{
    CShape *s1=new CShape;//动态创建图形对象
    s1->area();//调用计算图形面积函数
    Circle  *c1=new Circle(2.0);//动态创建图形对象
    s1=c1;//图形指针s1指向圆指针c1指向的圆对象
    s1->area();//调用计算面积函数?
    CRetangle *r1=new CRetangle(2.0,3.0); //动态创建长方形形对象。
    s1=r1;//图形指针s1改为指向长方形指针r1指向的长方形对象
    s1->area();//调用计算面积函数?
    return 0;
}
```

计算面积：
计算面积：
计算面积：



★多态实现规则

西南交通大学
Southwest Jiaotong University

```
int main()
{
    CShape *s1=new CShape;//动态创建图形对象
    s1->area();//调用计算图形面积函数
    Circle  *c1=new Circle(2.0);//动态创建图形对象
    cout<<"圆面积:"; c1->area();
    s1=c1;//图形指针s1指向圆指针c1指向的圆对象
    s1->area();//调用计算面积函数?
    CRetangle *r1=new CRetangle(2.0,3.0); //动态创建长方形形对象。
    cout<<"长方形面积:"; r1->area();
    s1=r1;//图形指针s1改为指向长方形指针r1指向的长方形对象
    s1->area();//调用计算面积函数?
    return 0;
}
```

运行结果:

```
计算面积:
圆面积:12.56
计算面积:
长方形面积:6
计算面积:
```

★多态实现规则

分析

- 通过派生类的对象名.成员名或者对象指针->成员名的方式，可以访问到各派生类中新添加的同名成员。
- 根据赋值兼容原则，可以将派生类对象的地址赋值给基类的指针，但是通过基类的指针，只能访问到从基类继承的成员。

  程序运行过程中，分别把基类对象、派生类Circle的对象和派生类CRetangle的对象赋值给基类类型的指针s1，但通过指针s1，只能使用继承下来的基类成员。

- 无法通过基类类型的指针访问派生类中有的而基类中没有的成员函数。

2. 规则2—虚函数

虚函数：虚函数是**动态联编**的基础。虚函数是非静态的成员函数，经过派生后，在类族中就可实现运行过程中的多态。



★C++环境下的虚函数语法规则

1. 虚函数的定义

语法形式

```
virtual 函数类型 函数名(形参表)
{
        函数体
}
```

说明：

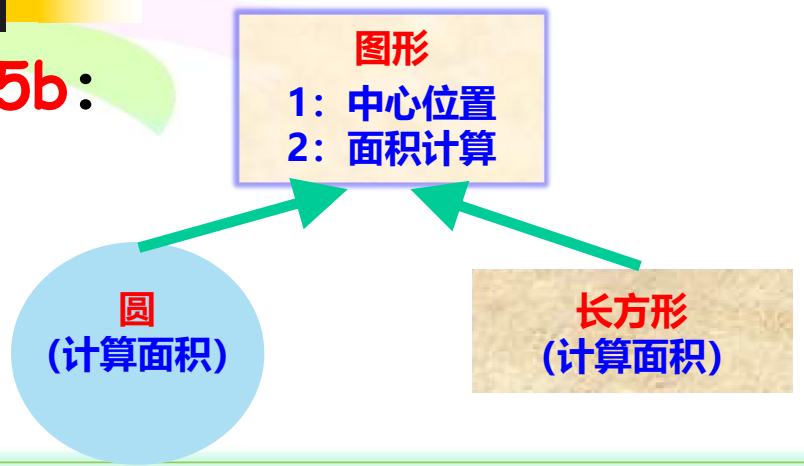
- 关键字virtual用于限定类的成员函数为虚函数，普通函数不能说明为虚函数。
- 关键字virtual只能出现在类声明中的函数原型声明中，在类外实现成员函数体时不能写virtual。



★C++环境下的虚函数语法规则

西南交通大学
Southwest Jiaotong University

例5b：




```
class Center
{
private:
    double x,y;
public:
    Center(){x=0,y=0;}
    ~Center(){cout<<"~中心"<<endl;}
    void SetCenter(double x1,double y1)
    {x=x1;y=y1;}
};
```

```
class CShape
{
private:
   Center c1;
public:
   virtual void area()
   {cout<<"计算面积:"<<endl;}
   virtual ~ CShape()
   {cout<< "~图形"<<endl;}
    void setC(double x1,double y1)
    {   c1.SetCenter(x1,y1);}
};
```

★C++环境下的虚函数语法规则


```
class Circle:public CShape
{
private:
     double radius;
public:
     Circle(double r):radius(r){}
     virtual ~Circle ()
     {cout<< "~圆 "<<endl;}
     virtual void area()//圆计算面积
     {
     cout<< 3.14*radius*radius<<endl;
     }
     void preimere()//圆计算周长
     {
     cout<< 2*3.14*adius<<endl;
     }
};
```

```
class CRetangle:public CShape
 {
  private:
       double width,height;
  public:
       CRetangle(double w1,double h1,double y)
       { width=w1; height=h1; }
       virtual ~CRetangle ()
       {cout<<"~长方形"<<endl;}
       virtual void area()//计算长方形面积
       {cout<<width*height<<endl;}
};
```

★C++环境下的虚函数语法规则


```
int main()
{
    CShape *s1=new CShape;//动态创建图形对象
    s1->area();//调用计算图形面积函数
    Circle  *c1=new Circle(2.0);//动态创建图形对象
    s1=c1;//图形指针s1指向圆指针c1指向的圆对象
    s1->area();//调用计算面积函数?
    CRetangle *r1=new CRetangle(2.0,3.0); //动态创建长方形形对象。
    s1=r1;//图形指针s1改为指向长方形指针r1指向的长方形对象
    s1->area();//调用计算面积函数?
    return 0;
}
```

```
计算面积:
12.56
6
```

加关键字virtual后的运算结果



★C++环境下的虚函数语法规则

2. 运行过程中的多态需要满足三个条件

① 类之间应该满足**赋值兼容规则**

② 要在成员函数上加**virtual**关键字声明**虚函数**

③ **动态绑定**：要由成员函数来调用或者是通过**指针**、**引用**来访问虚函数(因为如果是使用**对象名来访问虚函数**，则**联编在编译过程中就可以进行——静态联编**，因此不需要在运行过程中进行。)



★C++环境下的虚函数语法规则

3. 多态中虚函数判定标准

- 在基类设置了虚函数之后，通过基类类型的指针就可以访问到正在指向的对象的成员，这使得能够对同一类族中的对象进行统一的处理。
- 如果派生类并没有显式给出虚函数声明，这时系统就会遵循以下规则来判断一个函数成员是不是虚函数：
  - 名称：该函数是否与基类的虚函数有相同的名称；
  - 参数：该函数是否与基类的虚函数有相同的参数个数及相同的对应参数类型；
  - 返回值：该函数是否与基类的虚函数有相同的返回值类型。
- 由系统自动判断：若从名称、参数及返回值三个方面检查后，派生类的函数满足上述条件，就被自动确定为虚函数。

★多态的实现原理

1. 思考

- 多态的关键在于通过基类指针或引用调用一个虚函数时，编译时不能确定到底调用的是基类还是派生类的函数，运行时才确定——这叫动态联编。
- 动态联编底是怎么实现的呢?



★多态的实现原理

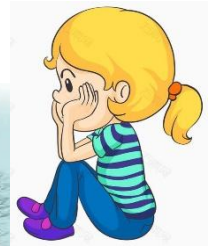
例07：

```
class Base
{
public:
     int i;
     virtual void Print() { cout << "Base:Print" ; }
};
class Derived : public Base
{
 public:
     int n;
     virtual void Print()
      {cout <<"Drived:Print" <<endl; }
};
int main()
{
     Derived d;
     cout << sizeof(Base) << endl;
     cout<< sizeof(Derived)<<endl;
     return 0;
}
```

加关键字virtual后的运算结果

```
8
12
请按任意键继续. . .
```

为什么都多了4个字节?



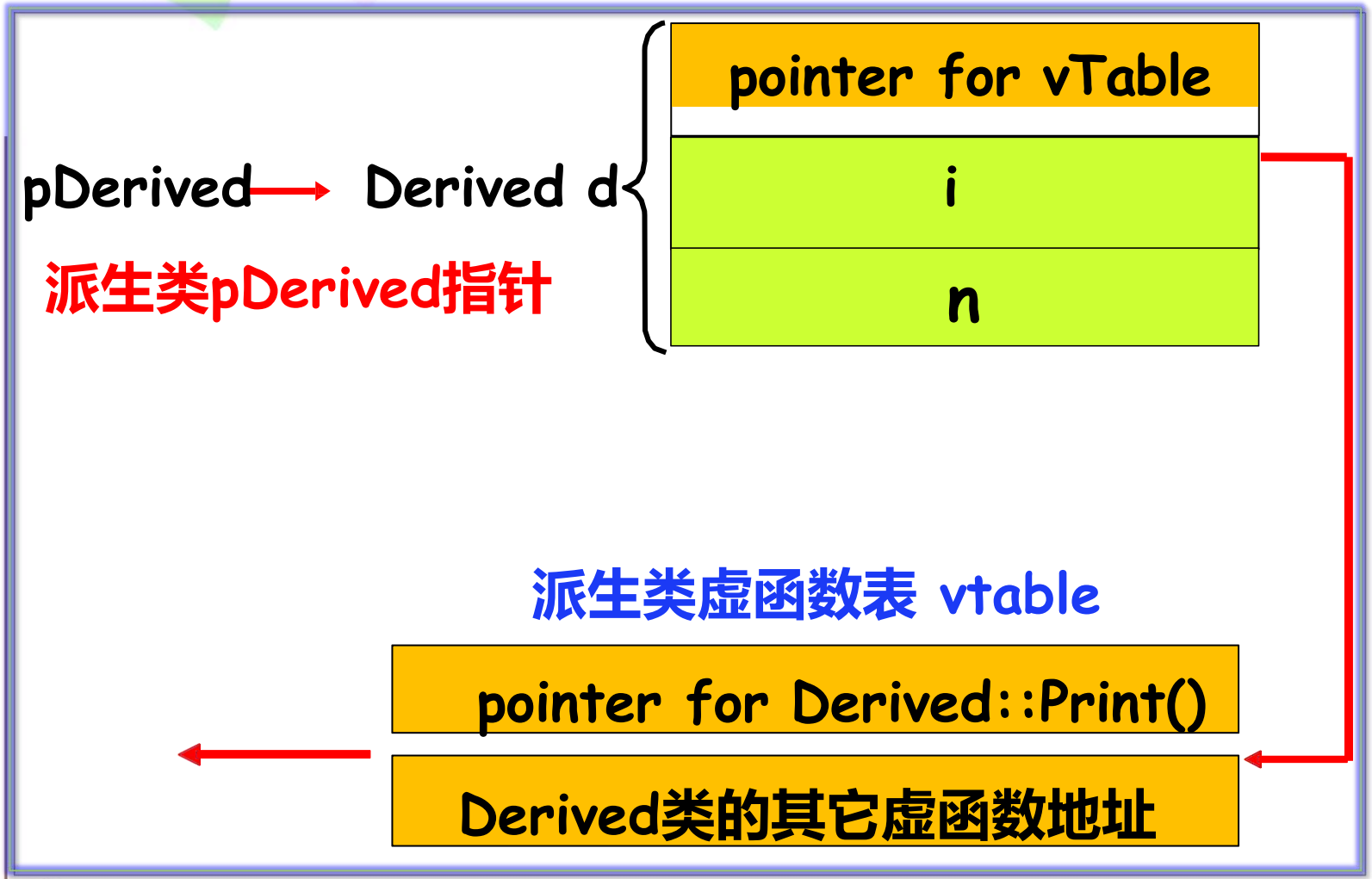
★多态的实现原理

2. 多态实现的关键—虚函数表

每一个有虚函数的类(或有虚函数的类的派生类)都有一个虚函数表。该类的任何对象中都放着虚函数表的指针。虚函数表中列出了该类的虚函数地址。多出来的4个字节就是用来放虚函数表的地址的。



★多态的实现原理



多态实现的关键—虚函数表

pBase = pDerived;

pBase->Print();

多态的函数调用语句被**编译成一系列**根据基类指针所指向的或基类引用所引用的**对象中存放的虚函数表的地址**，在虚函数表中 查找虚函数地址，并调用**虚函数的过程**。

★虚析构函数

例7a：未设置虚析构函数

```
class Base
 {public:
     ~Base() {cout<<"~Base "<<endl;}
};
class Derived : public Base
{public:

  ~Derived() {cout<<"~Derived "<<endl;}

};
int main(){
  Base *p = new Derived;
  delete p;
  return 0;
}
```

运行结果

~Base

- 通过基类的指针删除派生类对象时
  - → 只调用基类的析构函数
- 删除一个派生类的对象时
  - → 先调用派生类的析构函数
  - → 再调用基类的析构函数



★虚析构函数

解决办法

把基类的析构函数声明为virtual

- 派生类的析构函数 virtual可以不进行声明
- 通过 基类的指针 删除 派生类对象 时
  - → 首先调用 派生类的析构函数
  - → 然后调用 基类的析构函数

类如果定义了虚函数，则最好将析构函数也定义成虚函数

构造函数不能定义为虚函数



★虚析构函数

例7b：虚析构函数

```
class Base
 {public:
     Virtual ~Base() {cout<<"~Base "<<endl;}
};
class Derived : public Base
{public:
    virtual ~Derived() {cout<<"~Derived "<<endl;}
};
int main(){
    Base *p = new Derived;
    delete p;
    return 0;
}
```

运行结果

```
~Derived
~Base
```



★内容回顾

1. 多态、消息、联编、绑定……

   📖静态多态(编译时的多态)包括：函数多态、宏多态、模板多态

   📖动态多态(运行时的多态)

2. 多态的实现：静态联编、动态联编

3. 多态实现规则：满足赋值兼容规则、虚函数

   动态绑定：要由成员函数来调用或者是通过指针、引用来访问虚函数。基类类型的指针、虚函数

4. 多态实现的关键 —— 虚函数表

5. 虚析构函数

为什么要使用虚函数?



★使用虚函数时的注意事项

1. 构造函数和静态成员函数不能是虚函数

① 静态成员函数不能是虚函数，因为静态成员函数**没有this指针**，不受限制于某个对象；

② 构造函数不能是虚函数，因为构造的时候，对象还是一片未定型的空间，只有构造完成后，对象才是**具体类的实例**。



★使用虚函数时的注意事项

2. 构造函数或析构函数中调用虚函数

```
class son
{
public:
    virtual void hello()
    {cout<<"hello from son" <<endl;}
    son(){hello();};
    ~son(){bye();};
    virtual void bye()
    {cout<<"bye from son" <<endl;}
};
```

```
class grandson:public son
{
public:
    virtual void hello( )
    {cout<<"--hello from grandson" <<endl;}
    grandson( )
    {cout<<"--constructing grandson" <<endl;}
    ~grandson( )
    {cout<<"--destructing grandson" <<endl;}
};
```



★使用虚函数时的注意事项

构造函数或析构函数中调用虚函数

```
int main(){
    grandson gson;//先基类构造 后派生类构造，son()中静态调用hello()
    son *pson;//基类指针
    pson=&gson;//基类指针指向派生类对象
    pson->hello( );//动态调用 grandson::hello()
    return 0;
}
```

运行结果

```
hello from son
--constructing grandson
--hello from grandson
--destructing grandson
bye from son
```

在构造函数和析构函数中调用虚函数时：

被调用的函数是自己的类或基类中定义的函数，不会等到运行时才决定，编译阶段就确定了，即采用静态联编。





★使用虚函数时的注意事项

3. 普通成员函数中调用虚函数

```
#include <iostream>
using namespace std;
class Base
{
public:
    void func1()//普通成员函数
    { func2();}
    void virtual func2() //虚函数
    { cout << "Base::func2( )" << endl;}
};
class Derived :public Base
{
public:
    virtual void func2() //虚函数
    { cout << "Derived::func2( )" << endl;}
};
```

```
int main()
{
    Derived d;
    Base * pBase = & d;
    //调用普通成员函数
    pBase->func1( );
    return 0;
}
```

运行结果

```
Derived::func2( )
```



在普通成员函数中调用虚函数，则是**动态联编**，是**动态多态**。

★使用虚函数时的注意事项

4.私有虚函数和非虚接口(NVI)

```
#include <iostream>
using namespace std;
class Base
{
public:
    void func1()//普通成员函数
    { func2();}
private:
    void virtual func2() //私有虚函数
    { cout << "Base::func2( )" << endl;}
};
class Derived :public Base
{
private:
    virtual void func2() //私有虚函数
    { cout << "Derived::func2( )" << endl;}
};
```

```
int main()
{
    Derived d;
    Base* pBase = &d;
    //调用普通成员函数
    pBase->func1( );
    return 0;
}
```

Derived::func2( )

运行结果



好的程序设计规范：

接口和实现进行分离

★使用虚函数时的注意事项

5.有默认参数的虚函数

```
#include <iostream>
using namespace std;
class Base
{
public:
    void func1()//普通成员函数
    { func2();}
private:
    void virtual func2(int i = 2) //私有虚函数
    { cout << "Base::func2( ) " << i << endl;}
};
class Derived :public Base
{
private:
    virtual void func2(int i = 3) //私有虚函数
    { cout << "Derived::func2( ) " << i << endl;}
};
```

```
int main()
{
    Derived d;
    Base* pBase = &d;
    //调用普通成员函数
    pBase->func1( );
    return 0;
}
```

Derived::func2( )2

运行结果

虚函数是动态绑定的(即在运行时绑定)，但缺省参数是静态绑定的(即在编译时绑定)

强调 Emphasize

★纯虚函数和抽象类

1.纯虚函数：没有函数体的虚函数

纯虚函数：是一个在基类中说明的虚函数，在该基类中没有定义具体的操作内容，要求各派生类根据实际需要定义自己的版本。

语法形式

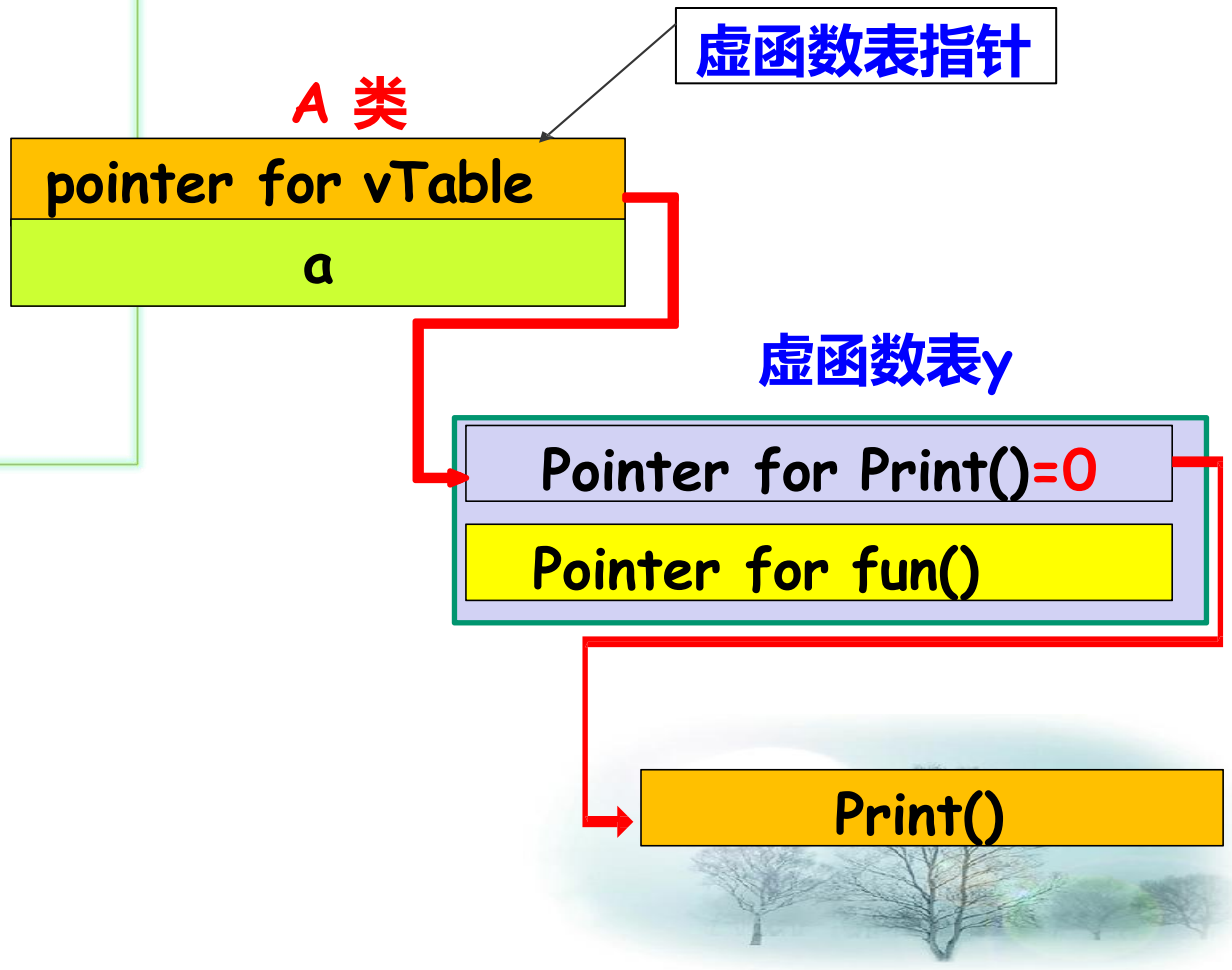
```
virtual 函数类型 函数名(参数表)=0;
```



★纯虚函数和抽象类

纯虚函数：没有函数体的虚函数

```
class A {
private:
    int a;
public:
    virtual void Print( )= 0 ;//纯虚函数
    //virtual void Print( ) { };//错误
    virtual void fun()
    {cout <<  "fun";}
};
```



★纯虚函数和抽象类

2.抽象类：包含纯虚函数的类

- 只能作为 基类 来派生新类使用
- 不能创建抽象类的对象
- 抽象类的指针和引用 ➔ 由抽象类派生出来的类的对象

```
A   a;          //错，A 是抽象类，不能创建对象
A*  pa;         //对，可以定义抽象类的指针和引用
pa = new A;     //错，A 是抽象类，不能创建对象
```

★纯虚函数和抽象类


```
class A//抽象类
{
public:
    virtual void f() = 0;
    //A( ){f();} //
    A() {}
    void g() { f(); }
};
class B : public A//非抽象类
{
public:
    void f()
    { cout << "B::f()" << endl;}
};
```

```
无法解析的外部符号 "public: virtual void __thiscall A::f(void)" (?f@A@@UAEXXZ)
```

```
int main()
{
    B b;
    b.g();
    return 0;
}
```

运行结果

B:: f()

强调 Emphasize

在抽象类中：

- 在构造函数/析构函数内部**不能**调用纯虚函数
- 在普通成员函数内可以调用纯虚函数
- 如果一个类从抽象类派生而来，必须在该派生类实现了基类中的所有纯虚函数，才能成为**非抽象类**

★纯虚函数和抽象类

抽象类：包含纯虚函数的类

- **抽象类**是一种**特殊的类**，它为一族类提供**统一的操作界面**。抽象类是为了抽象和设计的目的而建立的，即，建立抽象类，是为了通过它**多态的使用其中的成员函数**。
- 抽象类处于类层次的上层，一个**抽象类自身无法实例化**，也就是说**无法声明一个抽象类的对象**，而只能通过**继承机制**，**生成抽象类的非抽象派生类**，然后**再实例化**。
- **抽象类作用**：主要是通过抽象类为一个类族建立一个**公共的接口**，使它们能够更有效的发挥多态的特性。抽象类声明了一族派生类的共同接口，而接口的完整实现，即**纯虚函数的函数体**，要由派生类自己定义。



★纯虚函数和抽象类

3.接口类：仅含有纯虚函数的类

```
class CShape
{
public:
    virtual  void Area()=0;
    virtual ~ CShape(){}
    virtual  void  Length()=0;
};
```

实际运用中，接口类更多表达一种能力或协议。



★重谈：重载、覆盖、隐藏

1.成员函数被重载的特征

- 相同的范围(在同一个类中)
- 函数名字相同
- 参数不同
- 与virtual无关

2.覆盖是指派生类函数覆盖基类函数，其特征

- 不同的范围(分别位于派生类与基类)
- 函数名字相同
- 参数相同
- 基类函数必须有virtual关键字

覆盖规则其实就是C++**虚函数表**的实现原理



★重谈：重载、覆盖、隐藏

3.隐藏是指派生类的函数屏蔽了与其同名的基类函数

- 规则1：若派生类的函数与基类的函数同名，并且参数也相同，但是基类函数没有virtual关键字。此时，基类的函数被隐藏(注意别与覆盖混淆)
- 规则2：若派生类的函数与基类的函数同名，但参数不同，此时，不论有无virtual关键字，基类的函数将被隐藏(注意别与重载混淆)

隐藏规则实际上就是默认的C++名字查找过程



★重谈：重载、覆盖、隐藏

4. 如果基类有某个函数的多个重载(overload)版本，而你在派生类中重写(override)了基类中的一个或多个函数版本，或是在派生类中重新添加了新的函数版本(函数名相同，参数不同)，则所有基类的重载版本都被屏蔽，称之为隐藏hide。
5. 在派生类中要保持基类的函数重载版本，就应该重写所有基类的重载版本。

重写的模式：域作用分辨符::





★RTTI(Run-Time Type Information)

西南交通大学 Southwest Jiaotong University

1.静态语言和动态语言

动态语言、静态语言指动态类型语言(Dynamically Typed Language)和静态类型语言(Statically Typed Language)。

📖动态类型语言：在运行期间检查数据的类型的语言(如：python)

📖静态类型语言：在运行前编译时检查类型的语言(如：C++，java)。

```
function add(a,b)
{
    return a+b;
}
add(1,2);
add('1',2);
```

```
public int add(int a,int b)
{
    return a+b;
}
```

★RTTI(Run-Time Type Information)

2.C++的解决方案

RTTI：运行时类型检查(typeid()，dynamic_cast()强制转换运算符)。

- typeid是C++的一个关键字，主要作用就是让用户知道当前变量的实际类型。有两种语法形式
  - ① typeid (表达式); 如 typeid(5);
  - ② typeid (类型说明符); 如 typeid(int);
- 通过typeid得到的是一个type_info类型的常引用。
- type_info是C++标准库中的一个类，专门用于在运行时表示类型信息，定义在头文件typeinfo中。

★RTTI(Run-Time Type Information)


```cpp
#include<iostream>
#include<typeinfo>
using namespace std;

int main()
{
    int i1;
    cout<<typeid(100).name()<<endl;
    cout<<typeid(i1).name()<<endl;
    cout<<typeid(int).name()<<endl<<endl;
    cout<<typeid(double).name()<<endl;
    cout<<typeid(float).name()<<endl;

    cout<<typeid('f').name()<<endl;
    cout<<typeid("SWJTU126").name()<<endl;
    return 0;
}
```

获取类型名称

Dev cpp 运行结果

```
i
i
i

d
f
c
A9_c

--------------------------------
Process exited with return value 0
Press any key to continue . . .
```

VS2010 运行结果

```
C:\Windows\system32\cmd.exe
int
int
int

double
float
char
char const [9]
请按任意键继续. . .
```

★RTTI(Run-Time Type Information)

西南交通大学 Southwest Jiaotong University

```
#include <iostream>
#include <typeinfo>
using namespace std;
class A{
public:
    virtual void Print() { cout<<"This is class A."<<endl; }
};
class B : public A{
public:
    void Print() { cout<<"This is class B."<<endl;}
};
int main(){
    A *pA = new B();
    cout<<typeid(int).name()<<endl;
    cout<<typeid(pA).name()<<endl;       // class A *
    cout<<typeid(*pA).name()<<endl; // class A or class B?
    return 0;
}
```

注意：加virtual与不加区别

不加virtual运行结果

```
int
class A *
class A
```

加virtual运行结果

```
int
class A *
class B
```

获取类型名称：

typeid(int).name()

★RTTI(Run-Time Type Information)

3.static_cast

📖表达式：　　static_cast < Typeid > (expression)

📖static_cast：static_cast用来进行比较自然和低风险的转换，如整型和实数型、字符型之间互相转换。

static_cast：

- 不能在不同类型的指针之间互相转换
- 不能用于整型和指针之间的互相转换
- 不能用于不同类型的引用之间的转换



★RTTI(Run-Time Type Information)

运行结果：

西南交通大学 Southwest Jiaotong University

static_cast程序示例

```
#include <iostream>
using namespace std;
class A
{
public:
//重载了int转换运算符
   operator int()
   { return 1;}
};
class B :public A
{
public:
   void B_1() { cout << "B_1" <<
endl; }
};
```

```
int main()
{
   A a;
   int n1, n2;
   char* p = "Happy Birthday,SWJTU";
   B* b1 = static_cast<B*>(&a);
   //派生类指针指向基类对象，须显式转换
   b1->B_1();
   n1 = static_cast<int>(3.14);
   //n1 的值变为 3
   n2 = static_cast<int>(a);
   //调用a.operator int，n2的值变为 1
   //n = static_cast<int> (p);
   //error C2440 : "static_cast": 无法从 "char*" 转换为 "int"
   cout << n1 << "," << n2 << endl;
   return 0;
}
```

运行结果

```
B_1
3,1
```

★RTTI(Run-Time Type Information)

4. dynamic_cast

dynamic_cast <Type-id> (expression)

- Type-id必须是一个类的指针或引用，也可以是 void *
- 参数expression必须是一个能得到一个指针或者引用的表达式
- dynamic_cast：专门用于将多态基类(有虚函数的基类)的指针或引用，强制转换为派生类的指针或引用，且能够检查转换的安全性；对于不安全的指针转换，转换结果返回NULL指针。

表达式

dynamic_cast不能用于将非多态基类的指针或引用，强制转换为派生类的指针或引用



★RTTI(Run-Time Type Information)

西南交通大学 Southwest Jiaotong University

dynamic_cast程序示例

```
#include <iostream>
#include <typeinfo>
using namespace std;
class A
{
public:
    virtual void Print()
    {
        cout<<"This is class  A."<<endl;
    }
};
```

```
class B : public A
{
public:
    void Print()
    {cout<<"This is class  B."<<endl; }
    void B_1()
    {cout<<"B_1"<<endl;}
};
```

```
class C : public A
{
 public:
    void Print()
    {cout<<"This is class C."<<endl;}
    void C_1(){cout<<"C_1"<<endl;}
 };
```

★RTTI(Run-Time Type Information)

西南交通大学 Southwest Jiaotong University

dynamic_cast程序示例(续)

```
void Handle(A *a)
{
  B *b1; C *c1;
  if (typeid(*a)==typeid(B))
  //若指针a所指向的数据类型是B
  {//安全转换
     b1=dynamic_cast<B*>(a);
     b1->B_1();
  }
  if (typeid(*a)==typeid(C))
  {
     c1=dynamic_cast<C*>(a);
     c1->C_1();
  }
  a->Print();
}
```

```
int main()
{
     A *pA = new B();
     //基类指针指向派生类B的对象
     Handle(pA);
     delete pA;
     pA = new C(); //基类指针指向派生类C的对象
     Handle(pA);
     delete pA;
     return 0;
}
```

运行结果

```
B_1
This is class B.
C_1
This is class C.
```

★思考

1. 为什么类中有了虚函数之后,类的大小会增加 4 个字节呢?
2. 对多态的理解
   - ① (1)多态实现的条件
   - ② (2)多态实现的原理
3. 图片程序运行的结果及注释(1)原因

```
class A
{
    int x;
public:
    explicit A(int i) {}
    virtual void func()
    { cout << "A::a1" << endl;}
    void func2()
    { cout << "A::a2" << endl;}
};
```

```
class B
{
public:
    explicit B(int b):A(b) {}
    virtual void func()
    { cout << "B::b1" << endl; }
    void func2()
    { cout << "B::b2" << endl; }
};
```

```
int main()
{
    B b(1);
    //B d=1;  (1)为什么这里出现语法错误
    A& c = b;
    c.func();
    c.func2();
    return 0;
}
```